JavaScript

1. Introduction to JavaScript

JavaScript is a **high-level, interpreted, dynamically typed programming language** mainly used to build interactive and dynamic web applications.

- **High-level**: Developer does not manage memory directly; syntax is closer to human language.
- **Interpreted**: Code executes line by line; no separate compilation step (unlike Java, C++).

Dynamically Typed: Type is decided at runtime, not compile time.
```
let x = 10;

x = "hello"; // valid
```

Key Characteristics

- Runs in browser & server → Chrome (V8), Node.js
- Single-threaded → one call stack
- Asynchronous → handles non-blocking tasks
- Event-driven → reacts to user actions (clicks, inputs)

JavaScript vs HTML vs CSS

Tech	Role
HTML	Structure (skeleton)
CSS	Styling (appearance)
JavaScript	Behaviour (logic)

---

2. How JavaScript Works Internally

Execution Context: Created whenever JS runs code.

- **Memory Creation Phase**: Variables allocated, functions stored fully.
  - `var` → initialized as `undefined`
  - `let` / `const` → uninitialized (Temporal Dead Zone)
- **Execution Phase**: Code runs line by line, values assigned, functions executed.

Types of Execution Context

- Global Execution Context (created first)
- Function Execution Context (created on function call)

Call Stack

- Tracks execution contexts using **LIFO (Last In First Out)**.

Example:
function a() { b(); }

function b() { console.log("b"); }

a();

- Flow: Global EC → `a()` pushed → `b()` pushed → `b()` popped → `a()` popped.
- If stack is full → **Stack Overflow**.

---

3. Variables & Scope

- **var**: Function scoped, hoisted as `undefined`, can be redeclared.
- **let**: Block scoped, hoisted but in TDZ, cannot redeclare.
- **const**: Block scoped, must be initialized, cannot reassign (but object mutation allowed).

Scope Types

- Global → accessible everywhere
- Function → `var`
- Block → `let`, `const`

---

4. Data Types & Memory

- **Primitive (stored by value)**: number, string, boolean, undefined, null, symbol, bigint.
- **Non-Primitive (stored by reference)**: object, array, function.

👉 Interview Tip: Objects/arrays share references, so mutation affects all variables pointing to them.

---

5. Type Conversion & Coercion

Explicit Conversion:
Number("10"); // 10

String(100); // "100"

Boolean(0);   // false

-

Implicit Coercion:
"5" + 1; // "51"

"5" - 1; // 4

true + 1; // 2

-

Rule: `+` prefers string; other operators prefer number.

---

6. Operators

- `==` → compares values with coercion.
- `===` → compares values without coercion.
  👉 Interview Rule: Always use `===`.

---

7. Control Flow

- **if/else**: For boolean conditions.

switch: For multiple exact matches.

```
switch(role) {

  case "admin": break;

  case "user": break;

}
```

-

---

8. Loops

- **for**: Known iterations.
- **while**: Unknown iterations.
- **for...of**: Iterates values in arrays.
- **for...in**: Iterates keys in objects.

---

9. Functions

- **Declaration**: Hoisted, can be called before definition.
- **Expression**: Not hoisted, calling before definition → error.
- **Arrow Functions**: Short syntax, no `arguments`, no own `this`.

---

10. `this` Keyword

- Depends on **call site**, not definition.
  - `obj.fn()` → `this = obj`
  - `fn()` → global/undefined
  - `arrowFn()` → lexical `this`

---

11. Closures

Functions that remember variables from their outer scope.
Used for: data hiding, counters, memoization.

---

12. Arrays

- **map**: Returns new array.
- **filter**: Returns filtered array.
- **reduce**: Returns single value.
- **forEach**: Executes callback, returns `undefined`.

---

13. Objects

- Copy object: `const copy = {...obj}`
- **Shallow vs Deep Copy**: Shallow copies references; deep copies values.

---

14. Call, Apply, Bind

- `fn.call(obj, a, b)`
- `fn.apply(obj, [a, b])`
- `fn.bind(obj)` → returns new bound function.

---

15. Asynchronous JavaScript

- **Callbacks** → can lead to callback hell.
- **Promises** → states: pending, fulfilled, rejected.
- **async/await** → cleaner promise handling.

---

16. Event Loop

Execution priority:

1. Call Stack
2. Microtask Queue (Promises)
3. Callback Queue (`setTimeout`)

---

17. Error Handling

try { ... } catch(e) { ... } finally { ... }

---

18. DOM Manipulation

Used for UI updates, form handling, event binding.

---

19. Browser Storage

Storage	Size	Persist
localStorage	~5MB	Yes
sessionStorage	~5MB	No
cookies	~4KB	Yes

---

20. ES6+ Features

- Destructuring: `const {name} = user;`
- Spread: `const arr2 = [...arr];`

21.Higher-Order Functions (HOFs)

A **higher-order function** is one that either:

- Takes another function as an argument, or
- Returns another function as its result.

They make code **modular, reusable, and expressive**.

1. `map()`

- **Purpose**: Transform each element of an array.
- **Returns**: A new array.

```
const nums = [1,2,3];

const squares = nums.map(n => n*n);

console.log(squares); // [1,4,9]
```

2. `filter()`

- **Purpose**: Select elements based on a condition.
- **Returns**: A new filtered array.

```
const nums = [1,2,3,4];

const evens = nums.filter(n => n%2===0);

console.log(evens); // [2,4]
```

3. `reduce()`

- **Purpose**: Reduce array to a single value (sum, product, aggregation).
- **Returns**: A single value.

```
const nums = [1,2,3,4];

const sum = nums.reduce((acc,cur)=>acc+cur,0);

console.log(sum); // 10
```

`.forEach()`

- **Purpose**: Execute a function for each element.
- **Returns**: `undefined`.

```
[1,2,3].forEach(n => console.log(n*2));

// 2, 4, 6
```

👉 **Interview Tip**:

- Use `map` when you need a transformed array.
- Use `filter` when you need a subset.
- Use `reduce` when you need a single result.
- Use `forEach` for side effects (logging, mutations).

---

22. Debouncing

Debouncing is a **performance optimization technique** used to limit how often a function executes, especially for events like typing, scrolling, or resizing.

- **Definition**: Ensures a function runs **only after a certain delay** since the last call.
- **Use case**: Search boxes, window resize, button clicks.

```
function debounce(fn, delay) {

  let timer;

  return function(...args) {

    clearTimeout(timer);

    timer = setTimeout(() => fn.apply(this, args), delay);
```

```
  };
}

// Example: search input
const handleSearch = debounce(() => console.log("Searching..."), 300);
document.getElementById("search").addEventListener("input", handleSearch);
```

👉 **Interview Line**: “Debouncing prevents a function from firing too frequently by waiting until the user stops triggering the event.”

---

23. Event Propagation

Event propagation defines how events travel through the DOM. It has **three phases**:

1. **Capturing Phase**: Event travels from root → target.
2. **Target Phase**: Event reaches the target element.
3. **Bubbling Phase**: Event bubbles back from target → root.

```
document.getElementById("child").addEventListener("click", () => {
  console.log("Child clicked");
}, true); // capturing

document.getElementById("child").addEventListener("click", () => {
  console.log("Child clicked");
}); // bubbling
```

- **Default**: Bubbling.
- **Stop propagation**: `event.stopPropagation()` prevents event from moving further.
- **Use case**: Nested elements with overlapping event listeners.

👉 **Interview Line**: “Event propagation allows events to be handled at multiple levels; capturing goes top-down, bubbling goes bottom-up.”

Great question — **illegal shadowing** is a subtle but important concept in JavaScript scope management, and it often comes up in interviews. Let’s break it down clearly:

---

24. What is Shadowing?

Shadowing happens when a variable declared in an inner scope (block/function) has the same name as a variable in an outer scope.

- The inner variable *shadows* the outer one, meaning the outer variable becomes inaccessible within that inner scope.

Example:

```
let a = 10;

{

  let a = 20; // shadows outer 'a'

  console.log(a); // 20

}

console.log(a); // 10
```

Here, the inner `a` shadows the outer `a`.

🔹 Illegal Shadowing

Illegal shadowing occurs when you try to shadow a variable declared with `let` or `const` using `var` in the same scope chain.
This is **not allowed** because `var` is function-scoped and ignores block boundaries, leading to conflicts.

Example:

```
let x = 10;

{

  var x = 20; // ❌ Illegal shadowing

}
```

This throws an error: **“SyntaxError: Identifier 'x' has already been declared”**.

🔹 Legal Shadowing

Shadowing is allowed if:

- Both variables are declared with `let` (block scope ensures separation).
- Or if the outer variable is `var` and the inner one is `let` (block scope prevents conflict).

Examples:

```
// Legal shadowing with let

let y = 10;

{

  let y = 20;

  console.log(y); // 20

}

console.log(y); // 10


// Legal shadowing: var outside, let inside

var z = 30;

{
```

```
  let z = 40;
  console.log(z); // 40
}
console.log(z); // 30
```

🔹 Interview Rule of Thumb

- **Illegal**: `let/const` outside → `var` inside.
- **Legal**: `var` outside → `let/const` inside, or `let` outside → `let` inside.

👉 Key line: *"Illegal shadowing happens when a block-scoped variable (`let`/`const`) is shadowed by a function-scoped variable (`var`) in the same scope chain."*

Would you like me to also add **examples of shadowing with functions and closures** (where it gets trickier in interviews)?

24. Common Interview Traps

- Hoisting misunderstanding
- `this` confusion
- Promise execution order
- Mutation vs immutability

Output-Based Questions

1. Hoisting + var

```
console.log(a);
var a = 10;
console.log(a);
```

Output

undefined
10

Explanation: `var` is hoisted and initialized as `undefined`. Assignment happens later.

---

2. let & Temporal Dead Zone

```
console.log(b);
let b = 20;
```

Output

ReferenceError

Explanation: `let` is hoisted but not initialized. Access before declaration → TDZ error.

---

3. Function Hoisting

```
hello();
function hello() { console.log("Hello"); }
```

Output

Hello

Explanation: Function declarations are fully hoisted.

---

4. Function Expression Hoisting

```
hello();
var hello = function () { console.log("Hello"); };
```

Output

TypeError: hello is not a function

Explanation: `hello` is hoisted as `undefined`. Calling `undefined()` → TypeError.

---

5. Scope Chain

```
let x = 10;
function test() { console.log(x); }
test();
```

Output

10

Explanation: Function looks for `x` in outer (global) scope.

---

6. Block Scope

```
{ let a = 5; var b = 10; }
console.log(b);
console.log(a);
```

Output

10
ReferenceError

Explanation: `var` is function/global scoped; `let` is block scoped.

---

7. Closures (Classic)

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
```

```
  }
}
const fn = outer();
fn();
fn();
```

Output

```
1
2
```

Explanation: Closure keeps `count` alive across calls.

---

8. Closure inside loop (var)

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

Output

```
3
3
3
```

Explanation: `var` has single shared binding. Loop finishes before timeout runs.

---

9. Closure inside loop (let)

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

Output

```
0
1
2
```

Explanation: `let` creates new binding per iteration.

---

10. this inside object

```
const obj = { name: "JS", getName() { console.log(this.name); } };
obj.getName();
```

Output

JS

---

11. this detached method

```
const obj = { name: "JS", getName() { console.log(this.name); } };
const fn = obj.getName;
fn();
```

Output

undefined (browser non-strict) / TypeError (strict mode)

Explanation: Detached function call → `this` = global/undefined.

---

12. Arrow Function & this

```
const obj = { name: "JS", getName: () => { console.log(this.name); } };
obj.getName();
```

Output

undefined

Explanation: Arrow functions use lexical `this`, not object.

---

13. Array Mutation

```
const arr = [1,2,3];
const newArr = arr;
newArr.push(4);
console.log(arr);
```

Output

[1,2,3,4]

Explanation: Arrays are reference types.

---

14. Spread Operator

```
const arr = [1,2,3];
const newArr = [...arr];
newArr.push(4);
console.log(arr);
console.log(newArr);
```

Output

[1,2,3]
[1,2,3,4]

---

15. map vs forEach

```
const result = [1,2,3].forEach(x => x*2);
console.log(result);
```

Output

undefined

Explanation: `forEach` does not return anything.

---

16. reduce

```
const sum = [1,2,3,4].reduce((acc,cur)=>acc+cur,0);
console.log(sum);
```

Output

10

---

17. Type Coercion

```
console.log("10" - 5);
console.log("10" + 5);
```

Output

5
105

---

18. Equality Trap

```
console.log(0 == false);
console.log(0 === false);
```

Output

true
false

---

19. Event Loop

```
console.log("start");
setTimeout(()=>console.log("timeout"),0);
Promise.resolve().then(()=>console.log("promise"));
console.log("end");
```

Output

start
end
promise
timeout

Explanation: Promises (microtask queue) run before `setTimeout` (callback queue).

---

20. try / catch

try { console.log(a); } catch(e) { console.log("error"); }

Output

error

---

21. const object mutation

const user = { name: "A" };
user.name = "B";
console.log(user.name);

Output

B

---

22. Default Parameters

function add(a=5,b=10){ return a+b; }
console.log(add(2));

Output

12

---

23. Rest Operator

```
function sum(...nums){ return nums.length; }
console.log(sum(1,2,3,4));
```

Output

4

---

24. Object Keys Order

```
const obj = { 2:"b", 1:"a", name:"js" };
console.log(Object.keys(obj));
```

Output

["1","2","name"]

Explanation: Numeric keys sorted first.

---

25. delete operator

```
const obj = { a:1, b:2 };
delete obj.a;
console.log(obj);
```

Output

{ b:2 }

Revise Output based Questions from these links:
-https://github.com/lydiahallie/javascript-questions
-https://github.com/surbhidighe/Javascript-Output-Based-Questions
-https://github.com/Devinterview-io/javascript-interview-questions
-https://github.com/BilgeGates/Js-Interview-Questions/tree/master